

Sistemi e reti

Controllo della congestione TCP

Dalla crescita della finestra a TCP Tahoe e TCP Reno

Documento di studio elaborato a partire da tre videolezioni sul controllo della congestione TCP. Il testo organizza e mette in relazione i concetti trattati, senza essere una trascrizione dei video.

6 settembre 2026

Indice

1	Introduzione	2
2	Controllo di flusso e controllo della congestione	2
3	Le grandezze fondamentali	2
4	Slow start e congestion avoidance	3
4.1	Slow start	3
4.2	Congestion avoidance	3
5	Come TCP interpreta la perdita	3
6	TCP Tahoe	4
7	TCP Reno	4
8	Confronto tra Tahoe e Reno	5
9	Esempio concettuale	5
10	Conclusioni	5

1 Introduzione

TCP è un protocollo orientato alla connessione e affidabile: segmenta i dati, ne controlla la consegna tramite riscontri (ACK) e ritrasmette ciò che risulta perso. Tuttavia, l'affidabilità non dipende soltanto dagli estremi della comunicazione. I router lungo il percorso dispongono di buffer limitati: se ricevono più traffico di quanto riescano a inoltrare, le loro code crescono, i ritardi aumentano e alcuni pacchetti vengono scartati.

Il *controllo della congestione* è quindi l'insieme di meccanismi con cui il mittente TCP adatta la propria velocità alla capacità effettivamente disponibile nella rete. I tre video di riferimento presentano il tema in una progressione naturale: i concetti generali e le finestre di controllo; la politica TCP Tahoe; l'evoluzione TCP Reno.

2 Controllo di flusso e controllo della congestione

È importante distinguere due problemi simili, ma non coincidenti.

Aspetto	Controllo di flusso	Controllo della congestione
Obiettivo	Non sovraccaricare il ricevente.	Non sovraccaricare la rete e i router intermedi.
Informazione principale	Spazio nel buffer del ricevente.	Segnali indiretti: ACK, ACK duplicati, timeout e perdite.
Variabile associata	Finestra ricevuta, <i>rwnd</i> .	Finestra di congestione, <i>cwnd</i> .

Il mittente può mantenere in rete una quantità di dati non riscontrati limitata dalla più piccola delle due finestre:

$$W_{\text{eff}} = \min(rwnd, cwnd). \quad (1)$$

La *rwnd* è annunciata dal ricevente e descrive il suo margine di ricezione. La *cwnd* è invece mantenuta dal mittente come stima prudenziale della capacità della rete. Anche quando il ricevente è pronto ad accettare molti dati, TCP non deve inviarne più di quanto la rete sembri in grado di trasportare.

3 Le grandezze fondamentali

- **MSS (Maximum Segment Size):** dimensione massima utile dei dati contenuti in un segmento TCP; è l'unità pratica con cui si esprimono gli incrementi di finestra.
- **cwnd (congestion window):** limite imposto dal mittente ai dati non ancora confermati dagli ACK.
- **ssthresh (slow-start threshold):** soglia che separa la fase di crescita rapida dalla fase di crescita prudente.
- **RTT (Round-Trip Time):** tempo impiegato da un segmento per raggiungere il destinatario e dal relativo ACK per tornare al mittente.

All'avvio della connessione il mittente non conosce ancora la capacità disponibile lungo il cammino. Per questo TCP inizia con una finestra contenuta e la aumenta osservando l'arrivo degli ACK. La velocità non viene dunque fissata una volta per tutte: viene regolata durante la trasmissione.

4 Slow start e congestion avoidance

4.1 Slow start

La fase di *slow start* è attiva quando $cwnd < ssthresh$. Nonostante il nome, la crescita è rapida: per ogni ACK ricevuto la finestra cresce approssimativamente di un *MSS*. In condizioni ideali, nell'arco di un RTT il numero di segmenti inviabili raddoppia:

$$1, 2, 4, 8, 16, \dots$$

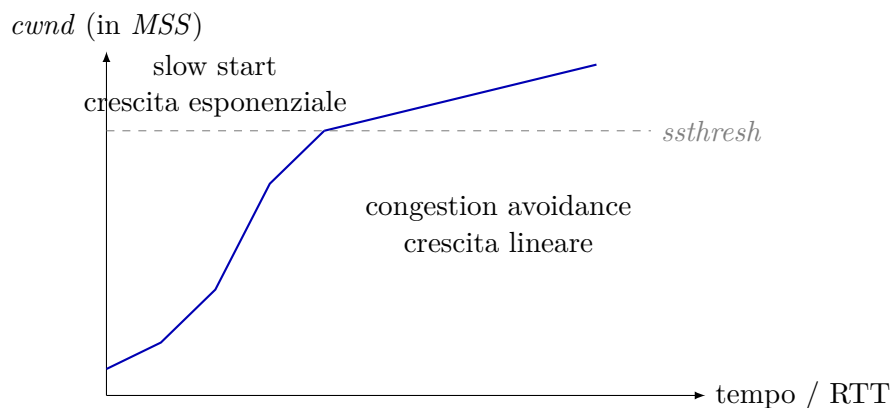
Questa strategia consente di sfruttare in breve tempo una rete inizialmente scarica, ma diventerebbe rischiosa se proseguisse senza limiti.

4.2 Congestion avoidance

Quando $cwnd \geq ssthresh$, TCP passa a *congestion avoidance*. La crescita diventa approssimativamente lineare: nell'insieme degli ACK di un RTT, $cwnd$ aumenta di circa un *MSS*. Una forma semplificata dell'aggiornamento per singolo ACK è:

$$cwnd \leftarrow cwnd + \frac{MSS^2}{cwnd}. \quad (2)$$

L'idea è aumentare ancora il traffico, ma con cautela, per ridurre la probabilità di saturare i buffer dei router. La soglia *ssthresh* è quindi lo spartiacque tra una fase esplorativa veloce e una fase di crescita controllata.



5 Come TCP interpreta la perdita

La perdita di un segmento è un segnale di possibile congestione, poiché può indicare che un router ha dovuto scartare dati a causa del riempimento della propria coda. TCP non può osservarne direttamente lo stato interno: inferisce il problema dai riscontri.

Due segnali sono particolarmente importanti:

- **Timeout:** non arriva l'ACK entro il tempo previsto. È un indizio forte di perdita o grave congestione.
- **Tre ACK duplicati:** il ricevente continua a confermare lo stesso numero di sequenza perché sono arrivati segmenti successivi, ma ne manca uno intermedio. È un segnale più circoscritto: la rete sta ancora consegnando dati, quindi non è necessariamente completamente congestionata.

Su questa differenza si basa la distinzione più significativa tra Tahoe e Reno.

6 TCP Tahoe

TCP Tahoe combina slow start, congestion avoidance e fast retransmit. Quando rileva la perdita — sia mediante timeout sia mediante tre ACK duplicati — applica una reazione conservativa:

$$\begin{aligned} ssthresh &\leftarrow \frac{cwnd_{precedente}}{2}, \\ cwnd &\leftarrow 1MSS. \end{aligned}$$

In forma semplificata, Tahoe imposta quindi *ssthresh* alla metà del valore che *cwnd* aveva prima del reset; subito dopo riporta *cwnd* a un solo *MSS* e riparte da slow start. Il mittente ritrasmette rapidamente il segmento mancante nel caso dei tre ACK duplicati. In altre parole, Tahoe tratta il segnale di perdita come indicazione che la velocità precedente non fosse sostenibile.

Vantaggio: è semplice e riduce con decisione il carico offerto alla rete. **Limite:** può essere eccessivamente prudente quando il percorso continua a consegnare i segmenti successivi e la perdita è isolata; la finestra crolla e occorre tempo per ricostruirla.

7 TCP Reno

TCP Reno conserva i meccanismi di base ma introduce *fast recovery*, con l'obiettivo di evitare che tre ACK duplicati causino sempre un ritorno completo a slow start. Il ragionamento è il seguente: se il ricevente invia ACK duplicati, sta comunque ricevendo segmenti fuori ordine; dunque esiste ancora capacità di trasferimento lungo il percorso.

Alla ricezione del terzo ACK duplicato, Reno esegue in modo semplificato questi passi:

1. imposta *ssthresh* alla metà del valore di *cwnd* precedente al reset;
2. ritrasmette immediatamente il segmento mancante (*fast retransmit*);
3. imposta $cwnd \leftarrow ssthresh + 3MSS$, tenendo conto dei tre segmenti che hanno generato gli ACK duplicati;
4. per ogni ACK duplicato aggiuntivo aumenta *cwnd* di un *MSS* e, se la finestra lo permette, può inviare un nuovo segmento;
5. quando arriva un ACK che conferma nuovi dati, imposta $cwnd \leftarrow ssthresh$ ed entra in congestion avoidance.

Con un timeout Reno resta invece prudente e ricade nel comportamento di riavvio da slow start. Fast recovery non significa ignorare la congestione: Reno dimezza comunque la soglia, ma evita di azzerare completamente la conoscenza utile acquisita sulla capacità del percorso.

8 Confronto tra Tahoe e Reno

Evento	TCP Tahoe	TCP Reno
Timeout	Riduce <i>ssthresh</i> , pone <i>cwnd</i> a 1 <i>MSS</i> e riparte in slow start.	Stessa reazione prudente.
Tre ACK duplicati	Fast retransmit, poi <i>cwnd</i> a 1 <i>MSS</i> e slow start.	Fast retransmit + fast recovery; la finestra non torna subito a 1 <i>MSS</i> .
Idea progettuale	Trattare ogni perdita come forte segnale di congestione.	Distinguere una perdita isolata da una congestione più grave.
Effetto sulle prestazioni	Recupero più lento ma molto conservativo.	Recupero più efficiente quando i segmenti successivi continuano ad arrivare.

9 Esempio concettuale

Si consideri un mittente che abbia $cwnd = 16MSS$. Un segmento va perso, mentre almeno tre segmenti successivi raggiungono il ricevente. Quest'ultimo continua a inviare ACK per il segmento mancante.

- Con **Tahoe**, dopo il rilevamento della perdita la soglia viene ridotta circa a $8MSS$, $cwnd$ torna a $1MSS$ e la crescita riprende esponenzialmente fino alla nuova soglia.
- Con **Reno**, la soglia viene anch'essa ridotta circa a $8MSS$, ma fast recovery consente di mantenere temporaneamente una finestra gonfiata di $8 + 3 = 11MSS$. Al riscontro che conferma la ritrasmissione, $cwnd$ viene portata a $8MSS$ e prosegue con crescita lineare.

L'esempio non sostituisce le regole complete di un'implementazione TCP reale, ma mette in evidenza l'idea chiave: Reno diminuisce il tasso di invio senza ricominciare inutilmente da zero quando il flusso degli ACK dimostra che la rete è ancora operativa.

10 Conclusioni

Il controllo della congestione trasforma TCP da semplice meccanismo di consegna affidabile a protocollo capace di condividere responsabilmente la rete. La finestra effettiva è limitata sia dal ricevente sia dalla rete; slow start e congestion avoidance regolano la crescita della trasmissione; la perdita porta a ridurre la velocità.

Tahoe rappresenta una risposta netta e prudente alla perdita. Reno aggiunge un'inferenza più raffinata: tre ACK duplicati suggeriscono che il cammino non è completamente interrotto. Grazie a fast retransmit e fast recovery, Reno conserva parte della finestra e migliora l'efficienza, pur continuando a ridurre la pressione sulla rete. Le versioni TCP moderne hanno introdotto ulteriori affinamenti, ma questi due modelli restano fondamentali per comprendere il principio di aumento graduale e riduzione moltiplicativa che caratterizza il controllo della congestione.

Riferimenti bibliografici

- [1] Roberto Manfrin, *Controllo della Congestione TCP*, YouTube.
<https://www.youtube.com/watch?v=pZvUfY1yDrg>
- [2] Roberto Manfrin, *Politiche Gestione Congestione TCP Tahoe*, YouTube.
<https://www.youtube.com/watch?v=WTNSWPJXPfk>
- [3] Roberto Manfrin, *Politiche Gestione Congestione TCP Reno*, YouTube.
<https://www.youtube.com/watch?v=9k7h-yu74ac>
- [4] M. Allman, V. Paxson, E. Blanton, *TCP Congestion Control*, RFC 5681, 2009.
<https://doi.org/10.17487/RFC5681>